Our Final Architecture (with CloudFront)

User (Browser)
↓
CloudFront CDN
↓
Public Application Load Balancer (Frontend ALB)
↓
Frontend EC2 Instances (Nginx + Static UI)
↓
Internal Application Load Balancer
↓
Backend EC2 Instances (PHP API)
↓
Amazon RDS MySQL (citymart database)

This is a 3-tier architecture enhanced with a CDN.

How it works

1. **Users / Internet**

Users access our application through:

https://citymart.samuelonyejekwe.com

or

https://d34nv3nql5vjj2.cloudfront.net

Requests first reach CloudFront.

2. **CloudFront (Content Delivery Network)**

CloudFront acts as:

• Global edge network
• Content cache
• security layer

Responsibilities:

A. Reduce latency

CloudFront caches content in edge locations worldwide, so users receive content from the nearest location.

B. Reduce load on our servers

Static resources like:

/assets/img/*
/assets/js/*
/assets/css/*

can be cached at edge locations.

C. Secure entry point

CloudFront terminates HTTPS for users.

Connection:

User → HTTPS → CloudFront

3. <u>Origin (Frontend ALB)</u>

CloudFront forwards requests to its origin:

citymart-frontend-alb-1505003275.eu-west-2.elb.amazonaws.com

This is our public Application Load Balancer.

Connection:

CloudFront → HTTP → Frontend ALB

We intentionally used HTTP only between CloudFront and ALB to avoid SSL hostname mismatch.

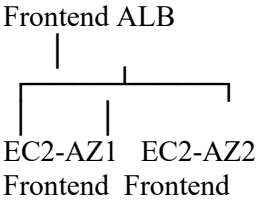
4. <u>Frontend ALB (Web Tier Load Balancer)</u>

The frontend ALB distributes traffic across our frontend servers.

Responsibilities:

• high availability
• load balancing across AZs
• health checks for frontend instances

Architecture:



5. Frontend EC2 Instances (Web Tier)

These servers host:

Nginx
HTML
CSS
JS

Responsibilities:

• serve the CityMart frontend UI
• route API calls

Example request:

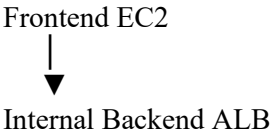
/products.html
/assets/js/products.js

When the frontend needs data, it calls:

/api/products

6. Internal Load Balancer (App Tier)

Frontend servers send API requests to an internal ALB.



Why internal?

Because backend servers are not exposed to the internet.

Security benefit:

Internet ❌ cannot reach backend servers

7. **Backend EC2 Instances (Application Tier)**

These servers run our PHP API.

Example endpoint:

/api/products

Responsibilities:

• business logic
• database queries
• API responses

Example response:

```
{
 "ok": true,
 "data": {
   "products": [...]
 }
}
```

8. **Amazon RDS (Database Tier)**

Our backend queries the RDS MySQL database.

Database:

citymart

Stored data:

products
orders
users

Connection:

Backend EC2 → RDS MySQL

Security Architecture

Our system uses multiple security layers:

Public access

Internet → CloudFront → Frontend ALB

Private access

Frontend EC2 → Internal ALB → Backend EC2

Database protection

Backend EC2 → RDS

The database is never publicly accessible.

High Availability

Our architecture is resilient because:

• EC2 instances run in multiple availability zones
• ALBs perform health checks
• Auto Scaling can replace unhealthy instances

Benefits of Adding CloudFront

CloudFront improves:

Performance

Edge caching reduces latency globally.

Scalability

Our servers handle fewer requests.

Security

CloudFront provides:

• HTTPS termination
• DDoS protection via AWS Shield
• integration with AWS WAF

Simple Flow of a Request

Example: user opens products page

User → CloudFront
CloudFront → Frontend ALB

Frontend ALB → Frontend EC2
Frontend EC2 → API call (/api/products)
Frontend EC2 → Internal ALB
Internal ALB → Backend EC2
Backend EC2 → RDS
Response travels back the same path

We implemented a highly available three-tier architecture on AWS with CloudFront acting as the CDN. CloudFront handles HTTPS and caches static content at edge locations, forwarding dynamic requests to a public Application Load Balancer in front of the web tier. The frontend EC2 instances serve the UI and communicate with backend API servers through an internal load balancer. The backend instances handle business logic and interact with an RDS MySQL database, which resides in private subnets. This design improves performance, scalability, and security."